MANIPAL UNIVERSITY JAIPUR

Department of Computer Science and Engineering
B.Tech. – Computer Science and Engineering
Specialisation: Data Science

Attention-Guided Residual Deep Convolutional Neural Network with Transfer Learning for Digital Art Style Classification

RESEARCH PAPER

Submitted in partial fulfilment of the academic requirements for
Computer Vision

Submitted By

Student Name	**Registration Number**
Aadhya Govil	2430010613
Anushka Dhanuka	2430010583
Divija Singh	2430010553

Project Supervisor: Dr. Ashish Pandey
Course: Computer Vision

October 2026

Declaration

We declare that this paper describes our work on a computer-vision project for classifying digital artworks by style. The cited publications are listed in the references. Dataset details and model settings are based on the supplied project scripts. Any performance values in the final version must come from the model's actual evaluation output; they should not be estimated or invented.

Aadhya Govil Registration Number: 2430010613
Signature: _____
Anushka Dhanuka Registration Number: 2430010583
Signature: _____
Divija Singh Registration Number: 2430010553
Signature: _____

Abstract

Online art collections contain many paintings, and sorting them by style can take a long time when done manually. Automatic classification could help people search, organise, and study these collections. The task is not simple: paintings from the same style can look different, while paintings from different styles can share colours, shapes, and textures.

This paper describes an attention-guided residual convolutional neural network that uses transfer learning. The implementation scripts use the WikiArt-10K dataset, resize images to 224×224 pixels, and use an ImageNet-pretrained ResNet-50 backbone. The attention module creates spatial weights and channel weights. These attended features are combined, passed through a residual block, and used to predict an artwork's style. The supplied training code also includes data augmentation, class-imbalance handling, two training stages, and a separate test-set evaluation script.

The project uses accuracy, macro F1-score, weighted F1-score, a classification report, and a confusion matrix to assess predictions. The uploaded materials include code for training and evaluation, but do not include a completed evaluation output with verified scores. Therefore, this paper does not claim a particular accuracy or that the model outperforms other models. Those conclusions should be added only after running the code and checking the saved results.

Keywords: WikiArt-10K, art-style classification, ResNet-50, CNN, attention, residual learning, transfer learning.

1 Introduction

Digital art collections are growing, and people need ways to search them efficiently. Artistic style is often recognised through features such as brushwork, colour, texture, shape, and composition. However, styles do not always have clear boundaries. A computer-vision model can learn patterns from labelled images and use them to predict a style for an image it has not seen before.

Convolutional Neural Networks (CNNs) are commonly used for image recognition because they learn simple visual patterns in early layers and more complex patterns in deeper layers . ResNet introduced shortcut connections that help train deep networks . Transfer learning starts from a model trained on a large dataset and adapts it to a new task . This can be useful when the target dataset is not large enough to train a deep network from scratch.

1.1 Problem statement

The goal is to predict one style label for each artwork image. The main difficulties are:

- paintings within one style can look quite different;
- different styles can share visual features;
- some style labels may have fewer images than others;
- a deep model can memorise training images instead of learning patterns that generalise.

1.2 Aim and objectives

The aim is to implement and evaluate an attention-guided residual CNN for artwork classification. The objectives are to prepare the dataset, use a pretrained ResNet-50, combine spatial and channel attention, train the classification head before fine-tuning selected backbone layers, and evaluate predictions on a held-out test set.

2 Related Work

2.1 CNNs and residual networks

CNNs apply learnable filters to local image regions. Early layers may respond to edges and colour changes, while later layers combine these patterns into more detailed features. AlexNet showed the strength of deep CNNs for large-scale image classification . VGG used repeated small convolution filters . ResNet uses shortcut connections so the input to a block can be added to the block's transformed output .

2.2 Attention mechanisms

Channel attention estimates which feature channels may be useful. Squeeze-and-Excitation networks use a summary of each channel to calculate its weight . Spatial attention instead assigns weights to locations in a feature map. CBAM combines both channel and spatial attention . In artwork images, these mechanisms may help the model emphasise patterns linked to colour, texture, or composition. This is a design motivation, not proof that the attention module improves accuracy.

2.3 Transfer learning and art classification

Transfer learning reuses features learned from a source task . The usefulness of these features depends on how different the source and target images are . Research has also explored automatic classification of fine-art paintings . The present project applies these ideas to a WikiArt-10K classification pipeline.

An important reference for this project is Zhang's study, "Improved Deep Convolutional Neural Network for Digital Art Image Classification and Identification" . The paper explores deep learning for classifying digital artworks into artistic styles and discusses attention mechanisms, residual connections, and transfer learning. These ideas are closely related to the approach examined in our project and provide useful background for understanding the proposed model.

2.4 Research gap

The individual techniques are established, but their combination must be tested on the chosen dataset. A fair evaluation should use fixed data splits, consistent preprocessing, and the same test data when comparing models. The supplied scripts define the main AAR model and a balanced-sampling experiment, but the actual scores are needed before drawing conclusions about which approach performs better.

3 Dataset and Data Preparation

3.1 Dataset source and labels

The supplied scripts identify the dataset as *WikiArt-10K*, stored as Parquet files. The target column is `style`. The preparation script reads the style labels and the dataset metadata, creates a mapping from original style IDs to consecutive model labels, and saves that mapping in `prepared_data/class_mapping.json`. The actual number of classes and image counts should be copied from the script output or generated metadata, rather than guessed.

3.2 Image validation and processing

The validation script checks whether image bytes can be opened and whether a style label is present. It records valid and invalid row indices and writes a CSV report. The processing script then reads valid images, converts them to RGB, resizes them to 224×224, and saves them as JPEG files. It also writes `processed_data/metadata.csv`, which records each image path, split, original style ID, model label, style name, and image dimensions.

3.3 Train, validation, and test sets

The preparation script uses a 70% training, 15% validation, and 15% test split with random state 42 and stratification by class. The training set is used to fit model weights. The validation set is used to monitor training and select checkpoints. The test set is kept for final evaluation.

Table 1: Dataset split defined in the preparation script

Subset	**Share**
Training	70%
Validation	15%
Testing	15%

The verification script checks the class mapping, required metadata columns, image dimensions, RGB mode, and whether dataset indices overlap across splits. These checks help catch common preparation errors. They do not by themselves prove that visually similar or duplicate paintings never occur across splits, so duplicate checking remains a useful extra step.

3.4 Preprocessing and augmentation

Images are resized to 224×224 and loaded as three-channel images. The model code applies ResNet-50 preprocessing. During training, the model uses random horizontal flipping, rotation, zoom, and translation. These changes are intended to make the model less dependent on the exact position of an image. Augmentation should be kept moderate so that it does not change the style label.

4 Proposed Model

4.1 Overall design

The model in the supplied training script is named `WikiArt_AAR_ResNet50`. It contains an ImageNet-pretrained ResNet-50 backbone without its original classification head. An attention module operates on the backbone feature map. A residual transformation then refines the attended features. Global average pooling and a dense classification head produce probabilities for the available style classes.

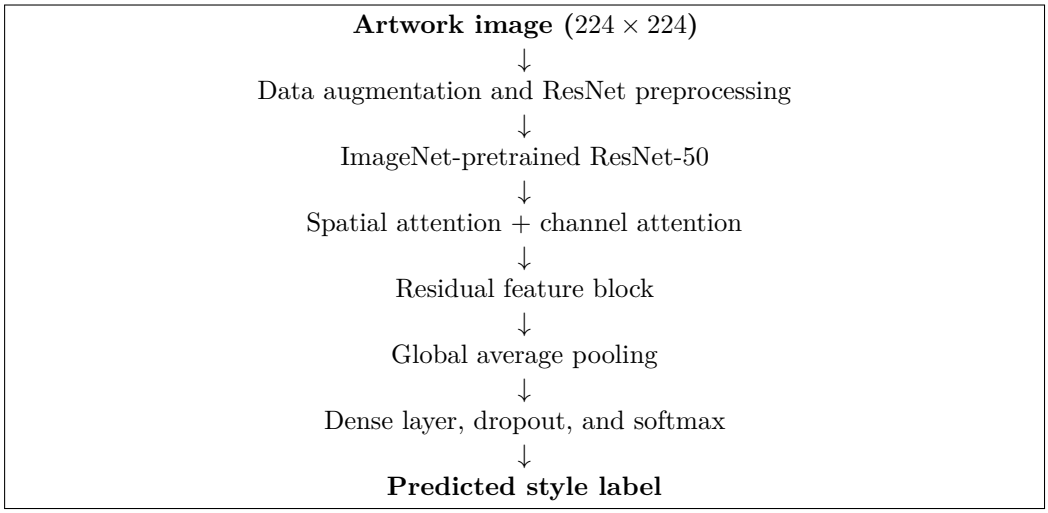


Figure 1: Model pipeline based on the supplied training script.

4.2 Spatial attention

Let F be the feature map produced by ResNet-50. The script uses a 1×1 convolution with sigmoid activation to create a spatial attention map:

$$M_S = \sigma(W_S * F + b_S), \qquad F_S = M_S \odot F.$$

The map gives a weight to each spatial location. Multiplication by the map changes the relative contribution of features at different locations.

4.3 Channel attention

The script applies global average pooling to F, then two dense layers to produce channel weights:

$$M_C = \sigma(W_2 \ \mathrm{ReLU}(W_1 z)), \qquad F_C = M_C \odot F.$$

The hidden layer size is set using the channel count and a reduction ratio of 16, with a minimum of 32 units. This gives the model a way to reweight feature channels.

4.4 Combining attention

The spatially attended and channel-attended features are added:

$$F_A = F_S + F_C.$$

The model then applies a residual transformation to F_A. The original backbone feature map is added to the transformed result, so the original representation has a direct path through the block. This is the residual part of the AAR design.

4.5 Residual block and classification head

The residual branch in the code uses a 1×1 convolution with 512 filters, batch normalisation, and LeakyReLU. A second 1×1 convolution returns the feature count to the backbone's channel dimension, followed by batch normalisation. The branch is added to the original feature map.

After that, global average pooling creates a vector. A 256-unit dense layer, batch normalisation, LeakyReLU, and dropout with rate 0.4 are used before the final softmax layer. The softmax output has one probability per style class. The predicted label is the class with the largest probability.

5 Training Method

5.1 Training settings

The scripts set the image size to 224, batch size to 8, random seed to 42, and use Adam. The classification-head stage has a starting learning rate of 10^{-3}. The fine-tuning stage uses 10^{-5}. Up to 25 epochs are allowed for the first stage and 20 for fine-tuning, although early stopping can end training sooner. These are the settings in the code, not a claim that they are optimal.

Table 2: Key settings in the supplied training script

Setting	**Value in script**
Dataset	WikiArt-10K
Backbone	ImageNet-pretrained ResNet-50
Input image size	224×224 RGB
Batch size	8
Random seed	42
Optimiser	Adam
Head learning rate	1×10^{-3}
Fine-tuning learning rate	1×10^{-5}
Maximum epochs	25 head + 20 fine-tuning
Dropout	0.4
Checkpoint selection	Lowest validation loss

5.2 Two training stages

In Stage 1, the ResNet-50 backbone is frozen while the attention/residual layers and classification head are trained. In Stage 2, the backbone is unfrozen and the last 30 layers are selected for fine-tuning, except batch-normalisation layers, which remain frozen. The lower learning rate is intended to limit large changes to pretrained features.

5.3 Training callbacks

The code uses model checkpointing to save the best model based on validation loss, early stopping to stop when validation loss stops improving, and learning-rate reduction when validation loss plateaus. CSV loggers save the training history. These files can be used to plot learning curves and check for overfitting.

6 Class Imbalance and Evaluation

6.1 Class imbalance

Some styles may have fewer examples than others. The main training script calculates class weights from the training labels and caps each weight at 5.0. The supplied alternative script tests balanced sampling: it samples training examples so each class has equal sampling probability, while keeping validation and test data unchanged. The balanced-sampling experiment saves its outputs in a separate folder so it does not overwrite the main model.

Balanced sampling can make rare classes appear more often during training, but it repeats some existing images. It does not create new artwork and may increase overfitting for small classes. The two approaches should be compared using the same held-out test set.

6.2 Evaluation measures

The evaluation script calculates the following measures:

- **Accuracy:** fraction of test images assigned the correct style.
- **Macro F1-score:** F1-score calculated separately for each class and then averaged equally across classes.
- **Weighted F1-score:** class-wise F1-scores averaged according to the number of true examples in each class.
- **Classification report:** class-wise precision, recall, F1-score, and support.
- **Confusion matrix:** counts of true and predicted labels for each class pair.

Accuracy is easy to understand but can hide weak performance on rare styles. Macro F1 gives each class equal importance, while weighted F1 gives more influence to classes with more test examples. The confusion matrix helps identify pairs of styles that the model often mixes up.

6.3 Evaluation procedure

The evaluation script loads the saved model and reads the existing split information from `metadata.csv`. It selects only rows marked `test`, loads each image, resizes it to 224×224, and obtains predictions. It then saves a classification report CSV, confusion matrix CSV, and a JSON summary under the models directory. The script evaluates an existing checkpoint; it does not retrain the model.

6.4 Fair comparison

The regular and balanced-sampling experiments should be compared only after both have been evaluated on the same test split. The same image preprocessing, class mapping, and metrics should be used. A single score is not enough to show that a model is better in every respect; class-wise results and training behaviour should also be considered.

7 Results and Discussion

7.1 Status of the results

The balanced-sampling evaluation was performed on 1,500 test images across 16 style classes. It achieved 68.73% accuracy, a macro F1-score of 0.5513, and a weighted F1-score of 0.6971. The comparison with class-weighted training is presented below. These values are reported from the supplied evaluation results.

Table 3: Final test performance — balanced sampling

Measure	**Observed value**
Number of test images	1,500
Number of style classes	16
Accuracy (%)	68.73%
Macro F1-score	0.5513
Weighted F1-score	0.6971

7.2 Regular versus balanced training

Table 4: Comparison of training strategies

Experiment	**Accuracy**	**Macro F1**
Class-weighted training (original model)	67.53%	0.5786
Balanced sampling	68.73%	0.5513

7.3 How to interpret the results

Balanced sampling achieved higher accuracy than class-weighted training (68.73% versus 67.53%), but its macro F1-score was lower (0.5513 versus 0.5786). This means balanced sampling made more test predictions correct overall, while class-weighted training had the better average F1-score when each style class was given equal weight. The balanced-sampling weighted F1-score was 0.6971. The results therefore show a trade-off between overall accuracy and macro F1 rather than one strategy being better on every measure.

The classification report should be checked for classes with low recall or precision. Low recall means many images of a style are missed; low precision means predictions for that style often belong to another class. The confusion matrix can show which pairs of styles are difficult to separate. Example images should be inspected before explaining why a specific error happened.

7.4 Limitations of the current evidence

The reported metrics summarise performance on the stated test split; they do not by themselves establish performance on other art collections. Class-wise precision, recall, F1-scores, and the confusion matrix should also be examined to identify which styles remain difficult to classify. The sample artwork image illustrates the visual content being considered, but one image is not enough to establish model performance.

8 Limitations and Future Work

The model relies on the labels and image quality available in WikiArt-10K. Style labels can be subjective, and the number of images may differ between classes. The model may also learn patterns related to image quality or collection-specific details instead of style itself. A held-out test set helps measure generalisation, but evaluation on a separate artwork collection would provide stronger evidence of transfer to new sources.

The attention module adds parameters and operations. Its value should therefore be assessed alongside computation time and memory use. Future work could compare the model with a plain pretrained ResNet-50 baseline, test different backbones, perform an ablation study, and inspect class-wise errors. The project could also add a small interface that lets a user upload an artwork and view the predicted style, but this should be described as completed only after it is implemented.

9 Conclusion

This paper describes an attention-guided residual CNN for digital art style classification using the WikiArt-10K data pipeline. The supplied code uses images resized to 224×224, an ImageNet-pretrained ResNet-50, spatial and channel attention, a residual refinement branch, and a softmax classification head. It includes a two-stage training procedure and an alternative experiment based on balanced class sampling.

On the balanced-sampling test run, the model achieved 68.73% accuracy, a macro F1-score of 0.5513, and a weighted F1-score of 0.6971 across 1,500 test images from 16 style classes. Compared with class-weighted training, balanced sampling achieved higher accuracy (68.73% versus 67.53%), while class-weighted training achieved a higher macro F1-score (0.5786 versus 0.5513). The comparison suggests that the choice of training strategy affects overall accuracy and class-balanced performance differently. Further class-wise analysis and evaluation on an independent artwork collection would help assess generalisation.

Reference

The main reference for this project is the research paper *Improved Deep Convolutional Neural Network for Digital Art Image Classification and Identification* by Huidong Zhang.

The paper explores how deep learning can be used to classify digital artworks into different artistic styles. It combines attention mechanisms, residual connections, and transfer learning to improve feature extraction and classification performance. These concepts provide the foundation for understanding the approach studied in our project.

H. Zhang, "Improved Deep Convolutional Neural Network for Digital Art Image Classification and Identification," *International Journal of Computational Intelligence Systems*, vol. 18, article 249, 2025. doi: 10.1007/s44196-025-00996-0.